

CSCI-GA.1170-003 Fundamental Algorithms

September 22, 2026

Problem Set 3

Lecturer: Julia Kempe

Due: 11:59pm on Monday, September 28

Problem 3-1 (Recurrences and Matrix Exponentiation) 16 points

The sequence $\{F_n \mid n \geq 0\}$ is defined as follows: $F_0 = 1$, $F_1 = 1$, $F_2 = 2$ and, for $i > 2$, define

$$F_i := F_{i-1} + 5F_{i-2} + 3F_{i-3}.$$

- (a) **(2 points)** Find a matrix A such that

$$A \cdot \begin{pmatrix} F_i \\ F_{i-1} \\ F_{i-2} \end{pmatrix} = \begin{pmatrix} F_{i+1} \\ F_i \\ F_{i-1} \end{pmatrix}.$$

- (b) **(3 points)** Use the equation from (a) and the ideas of fast $O(\log n)$ time exponentiation to build an efficient algorithm for computing F_n . Analyze its runtime in terms of the number of 3×3 matrix multiplications performed (each of which takes a constant number of integer additions/multiplications).
- (c) **(6 points)** Prove by induction that, for some constant $a > 1$, $F_n = \Theta(a^n)$. Namely, prove by induction that for some constant c_1 you have $F_n \leq c_1 \cdot a^n$, and for some constant c_2 you have $F_n \geq c_2 \cdot a^n$. Thus, F_n takes $O(n)$ bits to represent. What is the right constant a and the best c_1 and c_2 you can find?

Hint: Pay attention to the base case $n = 0, 1, 2$. Also, you need to do two very similar inductive proofs.

- (d) **(3 points)** In your algorithm of part (b) you only counted the number of 3×3 matrix multiplications. However, the integers used to compute (F_i, F_{i-1}, F_{i-2}) are $O(i)$ (in fact, $\Theta(i)$) bits long, by part (c). Thus, the 3×3 matrix multiplication used at that level of recursion will not take $O(1)$ time. In fact, using Karatsuba's multiplication, let us assume that the last matrix multiplication you use to compute (F_i, F_{i-1}, F_{i-2}) takes time $O(i^{\log_2 3})$. Given this more realistic estimate, analyze the actual running time $T(n)$ of your algorithm in part (b).
- (e) **(2 points)** Finally, let us look at the naive sequential algorithm which computes $F_3, F_4, \dots, F_n$ one-by-one. Assuming that each F_i takes $\Theta(i)$ bits to represent, and that integer addition/subtraction takes time $O(i)$ (multiplication by a constant can be implemented by a constant number of additions), analyze the actual running time of the naive algorithm. How does it compare to your answer in part (d)?

Problem 3-2 (Counting in a Grid)**23(+2) points**

Let us assume that you have an $m \times n$ matrix $M[1 \dots m][1 \dots n]$ where each row is sorted, in non-decreasing order, from left to right and each column is sorted, in non-decreasing order, from top to bottom. Values may repeat. The goal is to count the number of entries that are at most a particular value key . In other words, the goal is to find the number of indices (i, j) such that $M[i][j] \leq key$. For example, in the matrix

1	3	4	9
2	5	7	10
6	8	8	12
11	13	14	15

there are 9 entries that are at most $key = 8$.

- (a) **(2 points)** What are the indices of the largest element and the smallest element in M . Further, compare the elements $M[i][j]$ and $M[i+1][j+1]$. Justify your choices.
- (b) **(2 points)** Fill in the blanks to complete the brute force algorithm. What is the worst-case running time of the above algorithm?

```

1  GRIDCOUNT-A( $M, m, n, key$ )
2     $count = \dots\dots$ 
3    for  $i = \dots\dots$  to  $\dots\dots$ 
4      for  $j = \dots\dots$  to  $\dots\dots$ 
5        if  $\dots\dots\dots$  then  $count = \dots\dots$ 
6    return  $\dots\dots$ 

```

We will now look at possible ways of implementing divide and conquer to solve this problem. Specifically, let us look at a rectangular grid of size $m \times n$ where m and n are powers of 2. Let us consider an element $M[i][j]$ and the resulting quadrants as drawn below. Quadrant 1 contains $M[i][j+1]$, Quadrant 2 contains $M[i][j]$, Quadrant 3 contains $M[i+1][j]$ and Quadrant 4 contains $M[i+1][j+1]$, as depicted in the figure below.

QUADRANT 2	QUADRANT 1
$M[i][j]$	$M[i][j+1]$
$M[i+1][j]$	$M[i+1][j+1]$
QUADRANT 3	QUADRANT 4

We will compare key with $M[i][j]$. In the next question, you will identify which quadrant can be handled without any further exploration, and which quadrants still need to be counted.

- (c) **(4 points)** Clearly, identify the quadrant whose count is known without any further exploration, and the quadrants that still need to be counted, in the following two cases. Justify your choice.
- **if** $M[i][j] \leq key$
 - **if** $M[i][j] > key$

- (d) **(6 points)** Use part (c) to come up with a recursive algorithm based on divide and conquer strategy to solve this problem. In particular choose an appropriate i, j such that each sub-problem has the same number of elements when $m, n > 1$. Clearly, state the recursive calls made and how their results are combined. Identify a suitable base case.

Further, let $N = mn$ be the size of the sub-problem, and let $T(N)$ denote the running time of your algorithm on an input of size N . Formulate a recurrence relation for $T(N)$ in terms of the size of the subproblems, including its base case, and solve it to obtain the worst case running time.

- (e) **(6 points)** We can hope to achieve a significant speed-up by choosing $i = 1, j = n$. Immediately, you can notice that some of the quadrants defined earlier do not exist. Use the intuitions from part (c) to come up with the recursive algorithm. Use the hint provided to help you. Explain your algorithm and justify the correctness. Further, let $T(m, n)$ be the worst case running time of this algorithm on a grid of dimension $m \times n$. Write a recursive relation for $T(m, n)$ in terms of the subproblems and solve the same.

Hint: Focus on the quadrant(s) handled without further exploration in both cases of part (c). You might need to extend the boundary of one of these quadrants. In this strategy you can only hope to count or remove a maximum of n elements in one recursive call.

- (f) **(3 points)** Now count the number of entries *exactly equal* to *key* in $O(m + n)$ time. Your algorithm must be correct when *key* appears several times, including several times in the same row. Verify your algorithm on the example above with $key = 8$, where the answer is 2.
- (g) **(Extra Credit)(2 points)** Instead of your algorithm from part (e), we can also use binary search on each row. Briefly describe how to count the entries at most *key* this way in $O(m \log n)$ time, and compare it with your algorithm from part (e). When is each of the two approaches better?

Problem 3-3 (Counting Profitable Trades)

16(+4) points

You are given as input an array $A[1 \dots n]$ of stock prices, where $A[i]$ is the price on day i . In lecture, we computed the maximum profit $A[j] - A[i]$ over all $1 \leq i \leq j \leq n$. In this problem, we instead want to count how many pairs of days give a profit. A pair (i, j) is *profitable* if $i < j$ and $A[j] > A[i]$. Let $P(A)$ denote the number of profitable pairs. For example, if $A = \{4, 1, 3, 2, 5\}$, then $P(A) = 6$, since the profitable pairs are $(1, 5)$, $(2, 3)$, $(2, 4)$, $(2, 5)$, $(3, 5)$ and $(4, 5)$. Note that the prices need not be distinct, and a pair with $A[j] = A[i]$ is not profitable. For simplicity, you may assume that n is a power of 2.

- (a) **(1 point)** Describe how to compute $P(A)$ using $O(n^2)$ time.
- (b) **(4 points)** Let $L = A[1 \dots n/2]$ and $R = A[n/2 + 1 \dots n]$. Every profitable pair (i, j) falls into one of three cases: both i and j are in L , both are in R , or i is in L and j is in R . We call the pairs in the last case *cross pairs*. Show that the number of profitable cross pairs equals the number of pairs (l, r) with l in L , r in R and $l < r$, and conclude that sorting L does not change this count..

Use this to give a divide-and-conquer algorithm for computing $P(A)$, which counts the cross pairs by sorting L and performing a binary search for each element of R . Derive a recurrence relation for its running time with an appropriate base case, and solve it using the Master

Theorem. Do not forget to explicitly state the values of a , b , $f(n)$ and identify the correct case of the Master Theorem.

- (c) **(5 points)** The combine step in part (b) is slow because L is sorted again at every level of the recursion. As discussed in lecture for the stock profit problem, we can fix this by asking the recursive calls to return more information. Write pseudocode for a divide-and-conquer algorithm `COUNTANDSORT( $A$ )` that returns both $P(A)$ and a sorted copy of A in $\Theta(n \log n)$ time. Show that, when the two halves are returned sorted, the cross pairs can be counted while merging the two halves in $\Theta(n)$ time.

Hint: When an element of R is placed into the merged output, what do you know about the elements of L that have already been placed?

- (d) **(3 points)** Derive a recurrence relation for the running time of `COUNTANDSORT` with an appropriate base case, and solve it using the Master Theorem. How does it compare to the running times in parts (a) and (b)?
- (e) **(3 points)** Formally justify the correctness of `COUNTANDSORT` by induction, making sure that your algorithm handles repeated prices correctly.
- (f) **(Extra Credit)(4 points)** Instead of the number of profitable pairs, suppose we now want the *total profit* of all profitable pairs, i.e.,

$$\sum_{\substack{i < j \\ A[j] > A[i]}} (A[j] - A[i]).$$

For example, for $A = \{4, 1, 3, 2, 5\}$ the total profit is 13. Describe how to modify your algorithm `COUNTANDSORT` from part (c) to compute the total profit, still in $\Theta(n \log n)$ time. Justify the correctness and running time of your algorithm.

Hint: When an element of R is placed into the merged output, what else do you need to know about the elements of L that have already been placed?